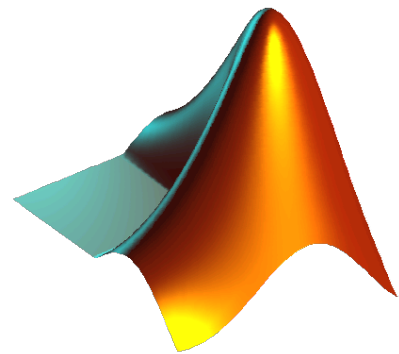


Rechnergestützte Numerik und Simulation

Übungsaufgaben



Übung 0: Aufgabe zum selber starten

Berechnen Sie mit Matlab eine Stammfunktion $F(x)$ von

$$f(x) = \sin(x) \quad \text{mit} \quad F(x) = \int_{x_l}^x f(x) dx$$

und stellen Sie $f(x)$ und $F(x)$ grafisch dar.

Übung 1: Einführung / Lineare Gleichungssysteme

Aufgabe 1.1 (ehemalige Klausuraufgabe)

Gegeben ist folgende Kette von Zuweisungen in MATLAB:

```
A = [1 2; 3 4];  
B = [1; 5];  
C = [1 3 4];  
C(7) = 2;  
D = [7; 4; 3; 2];
```

Überlegen Sie sich zunächst theoretisch das Ergebnis folgender Zuweisungen (falls sie funktionieren) und überprüfen Sie ihr Ergebnis am Rechner:

- a) `Ea = [A B];`
- b) `Eb = [A B; C];`
- c) `Ec = [A B; C(1:3)];`
- d) `Ed = [A(:,2), D(2:3); C(1:3:6)];`
- e) `Ee = [A(1:2); B'];`
- f) `Ef = [C(1:3); D(2:end)];`

Aufgabe 1.2 (ehemalige Klausuraufgabe)

Gegeben sind folgende Zuweisungen in MATLAB:

```
A = [1;2];  
B = [2,3];  
C = [1,2,3,4;5,6,7,8];  
D = [1,2;2,1;1,2;2,1];  
E(10,2) = 5;
```

Ermitteln und begründen Sie das Ergebnis folgender Zuweisungen, falls sie funktionieren:

- | | |
|----------------------------------|---|
| a) <code>Ea = [A,B'];</code> | b) <code>Eb = [C;D'];</code> |
| c) <code>Ec = [B,B;C];</code> | d) <code>Ed = [E(10,:),A];</code> |
| e) <code>Ee = A*B;</code> | f) <code>Ef = B*A;</code> |
| g) <code>Eg = B*C(:,2:3);</code> | h) <code>EH = C*A;</code> |
| i) <code>Ei = A(1,:) * E;</code> | j) <code>Ej = E(1:2,1:2) * E(5:6,1:2);</code> |

Aufgabe 1.3

Erzeugen Sie drei Zufallsmatrizen a , b , c der Dimension 5×5 und überprüfen Sie die Eigenschaften der Matrixmultiplikation in MATLAB anhand der folgenden Beispiele:

- $a*b$ verglichen mit $b*a$,
- $a*(b+c)$ verglichen mit $(a*b) + (a*c)$,
- $(a+b)*c$ verglichen mit $(a*c) + (b*c)$.
- a verglichen mit $\text{eye}(5)*a$ und $a*\text{eye}(5)$
- $\text{inv}(a)*a$ verglichen mit $a*\text{inv}(a)$ und $\text{eye}(5)$

Aufgabe 1.4

Lösen Sie das folgende lineare Gleichungssystem mit Hilfe von MATLAB:

$$x + y - z = 0; \quad 2x + y + z = 1 \quad x - 3z = -2$$

Aufgabe 1.5

Untersuchen Sie mit MATLAB, ob für quadratische Matrizen a , b die Rechenregel

$$\text{trace}(b*a*\text{inv}(b)) = \text{trace}(a)$$

gültig ist. Testen Sie dies mit Zufallsmatrizen. Beachten Sie, dass im Allgemeinen $b*a*\text{inv}(b) \neq a$, die Regel ist also alles andere als selbstverständlich. Hinweis: $\text{trace}(a)$ ist die Summe der Hauptdiagonalelemente von a .

Aufgabe 1.6

Folgendes Problem der linearen Algebra soll mit MATLAB gelöst werden:

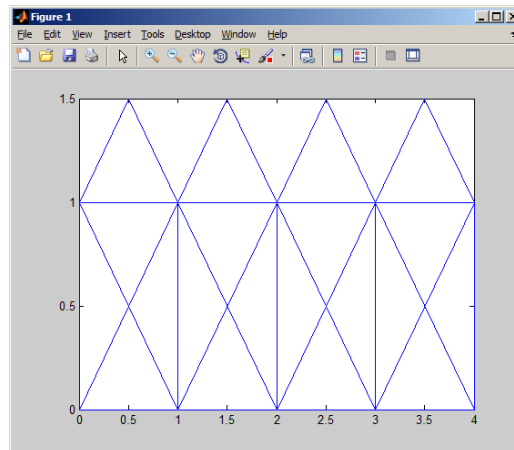
$$x \cdot A = x \cdot (b \cdot c - A) + c$$

mit $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, $b = \begin{pmatrix} 1 \\ 3 \end{pmatrix}$, $c = (1 \ 2)$.

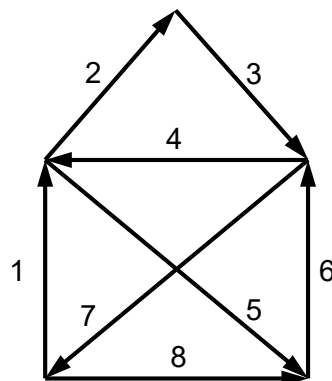
Schreiben Sie eine möglichst effiziente Sequenz, die x in MATLAB ermittelt.

Aufgabe 1.7

Schreiben Sie ein M-Skript, das das „Haus vom Nikolaus“ n -mal hintereinander zeichnet (s. Beispiel)! Nutzen Sie die Funktion von Matlab (insbesondere Array-Operationen) gezielt aus.



Für diejenigen die sich nicht mehr erinnern:



Aufgabe 1.8

Schreiben Sie ein M-Skript, das eine Matrix-Inversion nach dem Gauß-Jordan-Algorithmus durchführt, also elementare Zeilenumformungen ausnutzt, um die wie folgt dargestellte Umformung vorzunehmen:

$$\begin{pmatrix} 1 & 2 & 3 & | & 1 & 0 & 0 \\ 4 & 2 & 6 & | & 0 & 1 & 0 \\ 4 & 8 & 1 & | & 0 & 0 & 1 \end{pmatrix} \xrightarrow{\text{elementare Zeilenumformungen}} \begin{pmatrix} 1 & 2 & 3 & | & 1 & 0 & 0 \\ 0 & -6 & -6 & | & -4 & 1 & 0 \\ 0 & 0 & -11 & | & -4 & 0 & 1 \end{pmatrix}$$

$$\longrightarrow \begin{pmatrix} 1 & 0 & 0 & | & \dots & \dots & \dots \\ 0 & 1 & 0 & | & \dots & \dots & \dots \\ 0 & 0 & 1 & | & \dots & \dots & \dots \end{pmatrix}$$

Hinweis: Elementare Zeilenumformungen:

- Addieren des Vielfachen einer Zeile zu einer anderen.
- Vertauschen von Zeilen.

Aufgabe 1.9

Implementieren Sie die L-R-Zerlegung in Matlab (keine Anwendung der fertigen Matlab-Befehle) und lösen Sie das in Aufgabe 1.8 gegeben Gleichungssystem. Vergleichen Sie ihr Ergebnis mit dem des `lu`-Befehls bzw. des `linsolve`-Befehls unter Matlab.

Aufgabe 1.10

Gegeben ist folgendes Gleichungssystem:

$$\mathbf{Ax} = \mathbf{b} \quad \text{mit} \quad \mathbf{A} = \begin{bmatrix} 4 & -2 & 2 \\ 2 & -2 & 3 \\ 2 & 1 & 0 \end{bmatrix} \quad \text{und} \quad \mathbf{b} = \begin{bmatrix} -4 \\ 2 \\ 2 \end{bmatrix}$$

- Lösen Sie das Gleichungssystem mit dem Gauß-Algorithmus.
- Bestimmen Sie die L-R-Zerlegung von $\mathbf{A}$.
- Bestimmen Sie die maximale relative Abweichung in $\mathbf{b}$, wenn der Fehler $\mathbf{x}$ maximal 1% betragen darf.

Aufgabe 1.11

Berechnen Sie mit MATLAB die Inverse, die Determinante den von Rang von

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

Was fällt aus mathematischer Sicht auf? Erklärung?

Aufgabe 1.12

Überprüfen Sie ob

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 7 \\ 3 & 7 & 26 \end{bmatrix}$$

positive definit ist und bestimmen Sie die Cholesky-Zerlegung.

Übung 2: Funktionen / Iteratives Lösen von Gleichungen

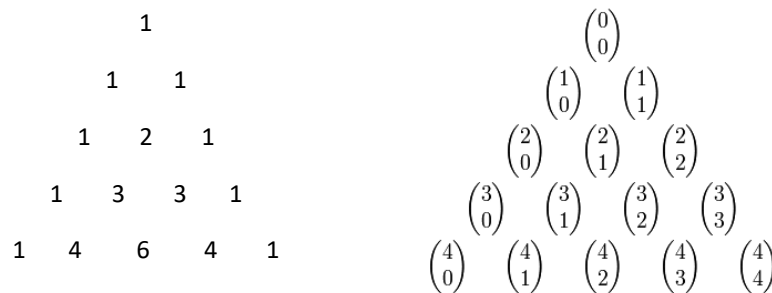
Aufgabe 2.1

Schreiben Sie eine rekursive Funktion zur Berechnung der Fakultät ($n!$).

Hinweis: $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (n-1) \cdot n$

Aufgabe 2.2 (ehemalige Klausuraufgabe)

Die Binomialkoeffizienten $\binom{n}{m}$ können z. B. durch das pascalsche Dreieck gebildet werden:



- Programmieren Sie in M eine rekursive Funktion die beliebige Binomialkoeffizienten $\binom{n}{m}$ nach dem Prinzip des pascalschen Dreiecks berechnet.
- Ist eine rekursive Funktion hier günstig? Begründen Sie Ihre Antwort!

Aufgabe 2.3 (ehemalige Klausuraufgabe)

Die Fibonacci-Zahlen F_n bilden die unendliche Reihe 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... mit

$$F_n = \begin{cases} 0 & \text{für } n = 0 \\ 1 & \text{für } n = 1 \\ F_{n-1} + F_{n-2} & \text{für } n > 1 \end{cases}$$

- Programmieren Sie in M eine rekursive Funktion die die Fibonacci-Zahl F_n berechnet.
- Ist eine rekursive Funktion hier günstig? Begründen Sie Ihre Antwort!
- Ergänzen Sie folgende Funktion (...) zur iterativen Berechnung der Fibonacci-Zahl F_n .

```

function f = fibonacci_i(n)
    ...
    for index = 1 : n - 1
        ...
    end
    ...
end
  
```

Aufgabe 2.4

Schreiben Sie eine Funktion in M, die ein vorgegebenes Verzeichnis und seine Unterverzeichnisse dem Matlab-Suchpfaden hinzufügt und folgende Eigenschaften hat:

1. Übergabewerte:
 - Verzeichnis (String) des betreffenden Unterverzeichnisses.
 - Optional: Tiefe mit der die Unterverzeichnisse berücksichtigt werden.
 - Optional: „Inversen“ Funktion (bereits enthalten Suchpfade werden entfernt).
2. Ausgabewerte:
 - Fehlerflag.
3. Zusätzliches:
 - Die Funktion soll (optional, s.o.) im Matlab-Fenster die zugefügten Suchpfade anzeigen.
 - Die Funktion soll die Eingaben überprüfen und eine sinnvolle Fehlerbehandlung haben.
 - Die Funktion soll einen sinnvollen Hilfetext anzeigen.

Aufgabe 2.5 (ehemalige Klausuraufgabe)

Beschreiben Sie kurz die Funktion der Suchpfadliste in Matlab?

- Wann kann eine von Ihnen geschriebenen Funktion aufgerufen werden?
- Was passiert, wenn zwei gleichnamige Funktionen in den Suchpfaden enthalten sind?
- Wie können Sie feststellen ob Namenskonflikte beim Funktionsaufruf existieren?

Aufgabe 2.6

a) Stellen Sie die Schnittpunkte der Funktionen

$$f(x) = \cos(x \cdot \sin(x)) \quad \text{und} \quad f(x) = x^2$$

in MATLAB grafisch dar. Und ermitteln Sie aus der Grafik die Werte der Schnittpunkte.

Hinweis: Der Befehl zum Erstellen eine x/y-Grafen lautet `plot(x,y)` mit den Vektoren `x` und `y`.

b) Stellen Sie in MATLAB eine Funktion dar, die identisch $\cos(x)$ ist, wenn der $\cos(x)$ größer 0 ist.

Aufgabe 2.7

Die Gleichung $x^3 - x + 0,3 = 0$ soll mit Fixpunktiteration ($x_0 = 0$) gelöst werden. Formulieren Sie das Problem um und prüfen Sie die Stabilität der Fixpunktiteration. Führen Sie eine Fehlerabschätzung durch.

Aufgabe 2.8

Implementieren Sie die **Regular-Falsi** in Matlab und lösen Sie mit Hilfe des Algorithmus die Gleichung $2 \cdot \sin(x) = x$.

Aufgabe 2.9

Lösen Sie mit Hilfe des Gesamtschrittverfahrens

$$\begin{pmatrix} 4 & -1 & 1 \\ -2 & 5 & 1 \\ 1 & -2 & 5 \end{pmatrix} \mathbf{x} = \begin{pmatrix} 1 \\ 2 \\ 0 \end{pmatrix} .$$

Überprüfen Sie vorab die Konvergenz.

Übung 3: Daten / Approximation von Funktionen

Aufgabe 3.1 (teilweise ehemalige Klausuraufgabe)

Folgende Zuweisung ist in MATLAB gegeben: `x = linspace(0,2*pi,100);`

Skizzieren Sie zeichnerisch den durch den `plot`-Befehle erzeugten Graphen und überprüfen Sie Ihr Ergebnis in MATLAB für:

- | | |
|---|------------------------------|
| a) <code>y1 = sin(x) .* (cos(x)>0);</code> | <code>plot(x,y1,'r');</code> |
| b) <code>y2 = boolean((sin(x+pi/2)<0)*3);</code> | <code>plot(x,y2,'m');</code> |
| c) <code>y3 = sin(x'*ones(1,length(x)))*sign(sin(x'));</code> | <code>plot(x,y3,'b');</code> |
| d) <code>y4 = cos(x) .* (sin(x)>cos(x));</code> | <code>plot(x,y1,'c');</code> |
| e) <code>y5 = (sin(x)>0)*3;</code> | <code>plot(x,y2,'k');</code> |

Aufgabe 3.2

- Legen Sie ein Struct-Array an, das eine Liste mit Studierenden, deren Matrikelnummern, Studiengang sowie zu jedem Studierenden eine Liste mit den belegten Fächern und den Noten verwaltet.
- Versuchen Sie mit den Elementen des Struct-Arrays zu arbeiten:
 - Zeigen Sie die vollständigen Daten eines Studierenden an.
 - Weisen Sie einem neuen Struct einen Studierenden als „Tutor“ zu.
 - Weisen Sie einem neuen Struct eine Gruppe von Studierenden als „Teilnehmer_MATLAB_Seminar“ zu.

Aufgabe 3.3 (ehemalige Klausuraufgabe)

Analysieren Sie folgende Funktion:

```
function out = structgen(varargin)
    for index = 1 : floor(nargin/2)
        switch varargin{index*2-1}
            case 'Name',
                out.Name = varargin{index*2};
            case 'Geburtsdatum',
                out.Geburtsdatum = varargin{index*2};
            case 'Beruf',
                out.Beruf = varargin{index*2};
        end
    end
end
```

Was wird als `person1`, `person2` und `person3` erzeugt, wenn folgende Funktionsaufrufe ausgeführt werden:

```
person1 = structgen('Name','Hans','Geburtsdatum','12.05.78','Beruf','Tischler');
person2 = structgen('Name','Geburtsdatum','12.05.78','Beruf','Tischler');
person3 = structgen('Name','Hans','Geburtsdatum','12.05.78',...
    'Beruf','Tischler','Name','Toni');
```

Aufgabe 3.4

Erzeugen Sie mit dem sprand-Befehl zwei schwach besetzte 100x100 Zufallsmatrizen. Stellen sie die Belegungsstruktur dar, berechnen Sie die Konditionszahl, die Eigenwerte und die Inverse der Matrizen sowie der des Produkts.

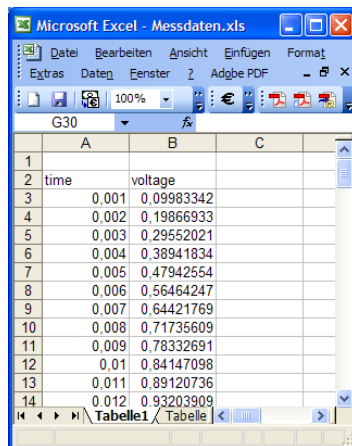
Stellen Sie außerdem fest wie viele Elemente in beiden Matrizen bzw. in mindestens einer belegt sind.

Aufgabe 3.5

Erstellen Sie ein xls-Datei oder alternative eine txt-Datei mit (frei erfundenen) Messdaten:

Aufsteigende **Zeitwerte** mit zugeordneten **Spannungswerten**.

Importieren Sie diese nach MATLAB und stellen Sie Messdaten grafisch dar.



	A	B	C
1			
2	time	voltage	
3	0.001	0.09983342	
4	0.002	0.19866933	
5	0.003	0.29552021	
6	0.004	0.38941834	
7	0.005	0.47942554	
8	0.006	0.56464247	
9	0.007	0.64421769	
10	0.008	0.71735609	
11	0.009	0.78332691	
12	0.01	0.84147098	
13	0.011	0.89120736	
14	0.012	0.93203909	

Beispiel:

Aufgabe 3.6 (ehemalige Klausuraufgabe)

Programmieraufgabe: Sortier-Algorithmen:

Bubble Sort: (einfachster Sortieralgorithmus)

Eine zu sortierende Liste von k Elementen wird vom ersten bis zum k -ten Element durchlaufen, wobei die jeweils benachbarten Elemente verglichen und gegebenenfalls vertauscht werden. Dies stellt beim ersten Durchlauf lediglich sicher, dass das jetzt letzte (k -te) Element an seinem korrekten Platz ist. Die Liste muss erneut bis zum $k - 1$ -Element verglichen werden, danach bis zum $k - 2$ -Element usw. Die Sortierung kann sofort abgebrochen werden, wenn bei einem Durchlauf keine Vertauschung mehr durchgeführt wurden.

Shuttle Sort: (etwas effizienter)

Auch hier wird die zu sortierende Liste von k Elementen vom ersten bis zum k -ten Element durchlaufen und die jeweils benachbarten Elemente verglichen und gegebenenfalls vertauscht. Im Unterschied zum Bubble-Sort-Algorithmus wird nach einer Vertauschung z. B. des m -ten und $m + 1$ -ten Elements anschließend rückwärts die Elemente m und $m - 1$, danach $m - 1$ und $m - 2$ usw. verglichen und gegebenenfalls vertauscht, allerdings nur solange wie eine Vertauschung notwendig

ist. Sind in rückwärtiger Richtung zwei Elemente in der richtigen Reihenfolge, so kann aufgrund der vorherigen Vergleiche davon ausgegangen werden, dass auch alle Elemente auf niederen Stellen in der richtigen Reihenfolge sind. Für die äußere Schleife ist ein Durchlauf ausreichend.

Die folgende Liste mit Namen ist als Cell-Array gegeben:

```
namensliste = {'Meier', 'Schmitt', 'Schulze', 'Brandt', 'Becker'};
```

Außerdem steht folgende Funktion zum alphanumerischen Vergleich von String-Arrays zur Verfügung:

```
function g = stralphacmp(str1, str2)

    n = min(length(str1), length(str2));
    g = sign(sum(((str1(1:n)>str2(1:n)) - ...
        (str1(1:n)<str2(1:n))) .* 2.^(n:-1:1)));
end
```

1. Analysieren Sie `stralphacmp`! Welche Ausgabe erfolgt aufgrund von:

```
stralphacmp('Meier', 'Schulze')
stralphacmp('Meier', 'Meier')
stralphacmp('Maier', 'Meierei')
stralphacmp('Becker', 'Meier')
```

2. Schreiben Sie eine in M eine Funktion die eine Liste von String-Arrays (wie die gegebene Namensliste) mittels des Bubble-Sort-Algorithmus sortiert.

Beispielaufruf:

```
namensliste_S = bubblesort(namensliste);
```

3. Schreiben Sie eine in M eine Funktion die eine Liste von String-Arrays (wie die gegebene Namensliste) mittels des Shuttle-Sort-Algorithmus sortiert.

Beispielaufruf:

```
namensliste_S = shufflesort(namensliste);
```

Aufgabe 3.7

Berechnen Sie die Newton'schen Interpolationspolynome zu den folgenden Daten

X_i	-1	0	1	2
F_i	-11	-2	3	22

Aufgabe 3.8

Die Funktion $\sin(x)$ soll im Intervall $[0, \pi/2]$ als stückweise lineare Interpolation mit äquidistanten Stützstellen approximiert werden. Der Fehler soll höchstens 10^{-5} betragen. Wie viele Stützstellen benötigt man.

Aufgabe 3.9:

Gegeben sind die folgenden Messdaten:

Zeit	0	1	2
Wert	0,5	3,5	6,5

Der Verlauf soll durch eine Gradengleichung der Form $f(t) = a \cdot t + b$ beschrieben werden. Bestimmen Sie durch Minimierung des Fehlerbetrags die Parameter a und b .

Aufgabe 3.10

Gegeben sei folgende Datenreihe:

	Punkt 1	Punkt 2	Punkt 3	Punkt 4	Punkt 5	Punkt 6
X	-6	-3	0	3	6	9
Y	-231	-61,5	-9	7,5	69	256,5

- Ermitteln Sie mit Matlab ein Polynom, welches die Messdaten möglichst gut approximiert. Berechnen Sie anschließend eine Reihe von Funktionswerten dieses Polynoms die 4-fach höher aufgelöst ist, als die vorliegende Datenreihe und stellen Sie die neue und alte Datenreihe grafisch gegenüber.
- Interpolieren Sie mit Matlab die oben angegebene Datenreihe mit 4-facher Auflösung und verschiedenen Methoden. Vergleichen Sie das Ergebnis mit dem aus Teil a.

Aufgabe 3.11

Schreiben Sie in MATLAB einen Algorithmus der das lineare Ausgleichsproblem mit zwei beliebigen Funktionen als Linearkombination berechnet. Überprüfen Sie den Algorithmus anhand eigener Messdaten.

Aufgabe 3.12

Schreiben Sie eine Funktion die ein Fuzzy-System auswertet.
Als Beispiel können Sie den Fuzzy-Regler aus dem Skript nutzen.

Aufgabe 3.13

Implementieren Sie unter Matlab ein eindimensionales RBF-Netz einschließlich einer Initialisierungs- und Trainingsfunktion und versuchen Sie damit die Funktion $f(x) = \sin(x)$ abzubilden. Vergleichen Sie das Ergebnis mit anderen Approximationsmethoden.

Aufgabe 3.14: (ehemalige Klausuraufgabe)

Ergänzen Sie die Funktion `polyfit_new` (...) (s. unten) so, dass diese den `polyfit`-Befehl gleichwertig ersetzt.

Hinweis: Der `polyfit`-Befehl ermittelt ein Polynom p so, dass die Summe E der Fehlerquadrate minimal wird:

$$E = \sum (y_n - p(y_n))^2 \rightarrow \min \quad \text{mit } p(y) \text{ Polynomfunktion}$$

Beispielaufruf:

```
x = [1 2 3];  
y = [3 5 2];  
pp = polyfit_new(x,y,3);
```

```
function p = polyfit_new(n,x,y)  
    ...  
  
    function e = summefehlerquadrat(p,x,y)  
        ...  
  
    end  
end
```

Übung 4: Grafik / M-Programmierung / Integration, Optimierung

Aufgabe 4.1

Schreiben Sie ein M-Skript, das die üblichen trigonometrischen Funktionen (Sinus, Cosinus, Tangens) im Bereiche von 0 bis π in einer gut formatierten Grafik (würdig einer Abschlussarbeit) zeichnet, mit

- Titel,
- Achsbeschriftungen,
- vernünftiger Skalierung.
- Pfeilen,
- unterschiedliche Linien und
- Legende.

Ergänzung:

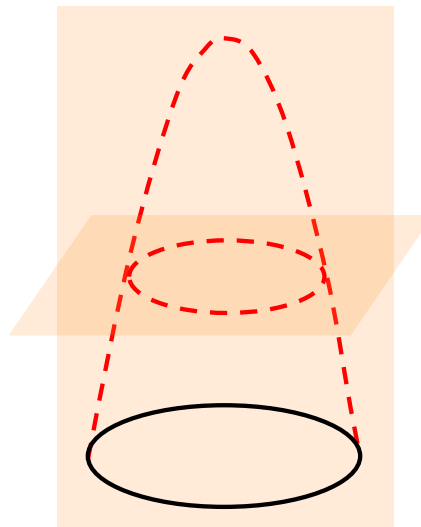
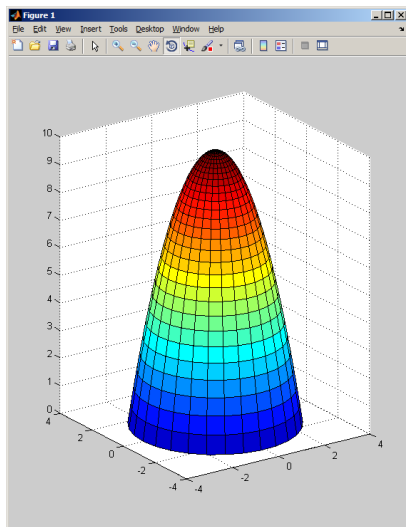
Erweitern Sie das Grafikfenster so, dass in vier separaten grafischen Darstellungen (Subplots) die trigonometrischen, die hyperbolischen sowie jeweils ihre Umkehrfunktionen dargestellt werden. Wählen Sie geeignete Achsskalierungen (linear, halb-logarithmisch oder logarithmisch).

Aufgabe 4.2

Erzeugen Sie (mit Phantasie- oder Zufallswerten) ein Diagramm über die Verteilung von weiblichen und männlichen Studierenden in unterschiedlichen Studiengängen! Wählen Sie selbst eine geeignet Form und Formatierung. Passen Sie die Grafikeigenschaften wie Schrift und Farben in geeigneter Form an.

Aufgabe 4.3

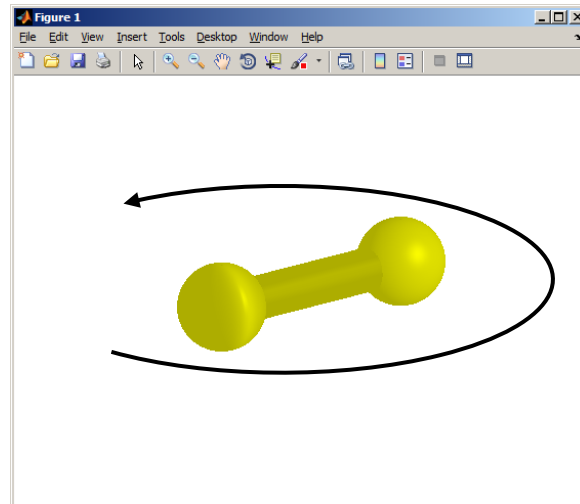
Schreiben Sie ein M-Skript, das einen Zuckerhut 3-dimensional grafisch darstellt (s. Bsp.). Hinweis: Der Längsschnitt durch den Zuckerhut soll eine Parabel sein, der Querschnitt ein Kreis.



Aufgabe 4.4

Schreiben Sie ein M-Skript, das eine „Hantel“ 3-dimensional grafisch darstellt und lassen Sie diese im Raum drehen.

Hinweis: Anstatt die Hantel zu drehen, kann man mit der „Kamera“ um das Objekt „herumfliegen“.

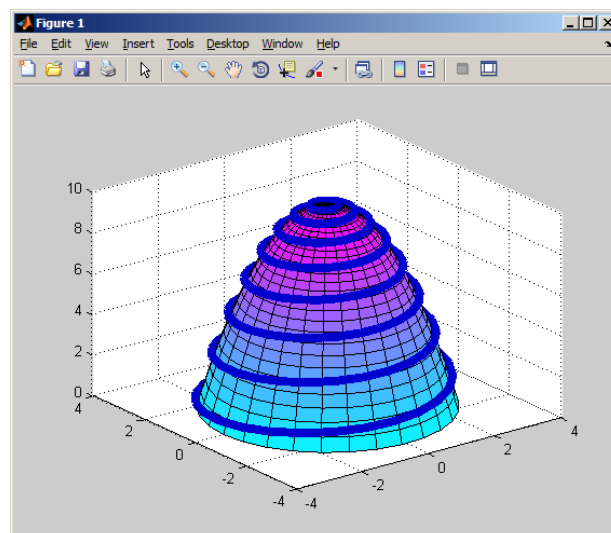


Aufgabe 4.5

Zeichnen Sie die trigonometrischen Funktionen aus Übung 4.2 mit den direkten Plot-Befehlen für Funktionen.

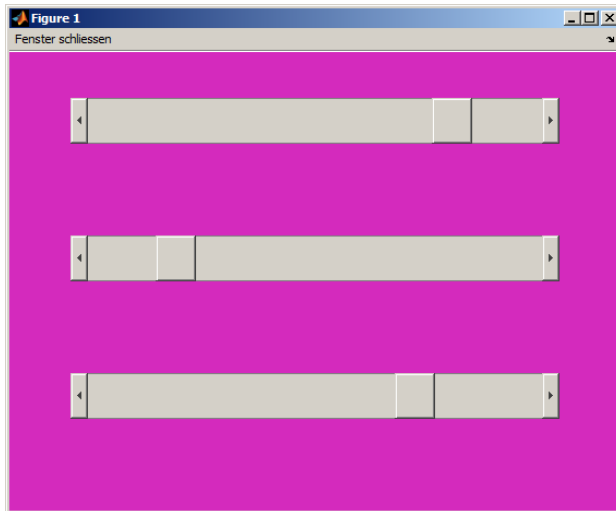
Aufgabe 4.6

Erweitern Sie das M-Skript aus Aufgabe 4.3 so, dass eine 3-dimensionale Helix um den Zuckerhut gelegt wird (s. Bsp.).



Aufgabe 4.7

Erzeugen sie ein GUI-Fenster, das drei „Slider“ für die RGB-Farben enthält, mit denen sich die Hintergrundfarbe des Fensters einstellen lässt.



Aufgabe 4.8 (ehemalige Klausuraufgabe)

Analysieren Sie folgende Funktion:

```
function out = funstruct(varargin)
    funhandle=@sin;
    result_struct='default';
    value = 0;
    for index = 1 : floor(nargin/2)
        switch varargin{index*2-1}
            case 'FunHandle',
                funhandle = varargin{index*2};
            case 'Value',
                value = varargin{index*2};
            case 'Struct',
                result_struct = varargin{index*2};
        end
    end
    result=feval(funhandle,value);
    eval(['out.',result_struct,'=result;']);
end
```

Was wird als `result1`, `result2` und `result3` bei folgenden Funktionsaufrufen erzeugt:

```
result1 = funstruct('FunHandle',@cos,'Value',pi,'Struct','cos.v');
result2 = funstruct('Value',pi);
result3 = funstruct(pi/2,'Value','Struct','sin.v');
```

Aufgabe 4.9

Schreiben Sie ein Script, das einen Kalendermonat übersichtlich, tabellarisch mit Wochentagen und Kalenderwochen im Kommandofenster von MATLAB anzeigt. Nutzen Sie die Formatierungsmöglichkeiten im Rahmen des `fprintf`-Befehls aus. Das Script soll flexible sein, um aus der Vorgabe „Erster Wochentag des Monats“, „Erste Kalenderwoche des Monats“ und „Anzahl der Wochentage des Monats“ jeden beliebigen Monat darzustellen. Beispiel:

	Mo	Di	Mi	Do	Fr	Sa	So
KW 39			1	2	3	4	5
KW 40	6	7	8	9	10	11	12
KW 41	13	14	15	16	17	18	19
KW 42	20	21	22	23	24	25	26
KW 43	27	28	29	30	31		

Aufgabe 4.10 (ehemalige Klausuraufgabe)

Schreiben Sie in M eine Funktion, die es ermöglicht beliebige andere Funktionen, die ansonsten nur skalare Argumente akzeptieren, elementweise auf Arrays (eindimensional) anzuwenden. Stellen Sie auch einen korrekten Funktionsaufruf dar.

Hinweis:

Gehen Sie von folgender Beispielfunktion aus die elementweise auf ein Array ausgeführt werden soll:

```

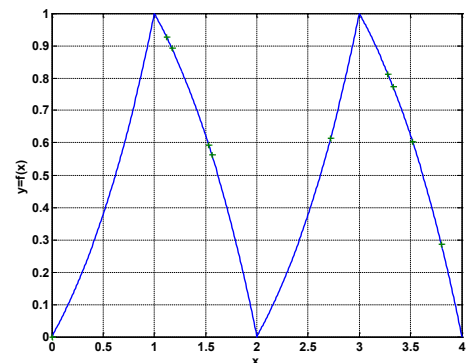
function out = testfun(in)
    if in > 0
        out = sin(in);
    else
        out = 0;
    end
end
  
```

Aufgabe 4.11

Gegeben ist die Sägezahnfunktion (`exp_fun.m`) im Intervall $[0,4]$ und 10 Messwerte.

x	0	1,12	1,17	1,53	1,56	2,72	3,28	3,33	3,52	3,8
Y	0	0,9258	0,8922	0,5932	0,5631	0,6137	0,8119	0,7725	0,6031	0,2868

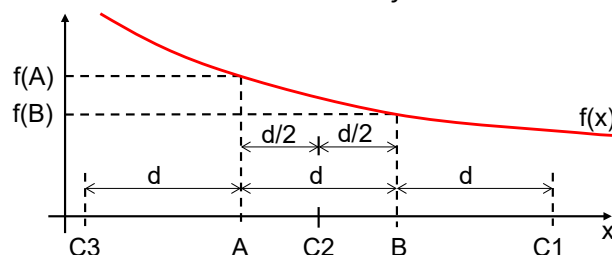
```
function y = exp_fun (x)
    k=fix(x)+1;
    norm=1/(exp(1)-1);
    if mod(k,2)~=0
        y=(exp(x-fix(x))-1)*norm;
    else
        y=(exp(1)-exp(x-fix(x)))*norm;
    end
end
```



- Bestimmen Sie die ungefähre Fläche unterhalb des Verlaufes anhand der Darstellung.
- Verwenden Sie die Datenreihe um den Verlauf mit dem `polyfit`-Befehl durch ein Polynom 6. Ordnung anzunähern und schreiben Sie ein Skript, welches das analytische Integral mithilfe der Stammfunktion des Polynoms (Polynom 7. Ordnung) berechnet.
- Integrieren Sie das Polynom mit dem `trapz`-Befehl und die Sägefunktion (`exp_fun.m`) mit dem `quad`-Befehl. Vergleichen Sie die Ergebnisse. Warum sind die Ergebnisse zwischen analytischer und numerischer Integration identisch, wenn der `quad`-Befehl auf das approximierte Polynom angewendet würde?

Aufgabe 4.12

Schreiben Sie eine Funktion in M die mittels des Nelder-Mead-Verfahrens nach dem Minimum einer beliebigen Funktion (1-dim.) sucht. Nutzen Sie Function-Handles als Übergabeargument Ihrer Suchfunktion. Hinweis: Rekursion des *Nelder-Mead-Verfahrens*:



- Starte an den Stützstellen A und B. Wenn $f(A) > f(B)$, dann gehe zu C1 (andernfalls (*) zu C3).
- Falls $f(C1) < f(A)$, (bzw. (*) $f(C3) < f(B)$), dann nehme B und C1 (bzw. (*) C3 und A) als neue Startwerte.
- Falls $f(C1) > f(A)$, (bzw. (*) $f(C3) > f(B)$), nehme C2 und B (bzw. (*) A und C2) als neue Startwerte.

Aufgabe 4.13:

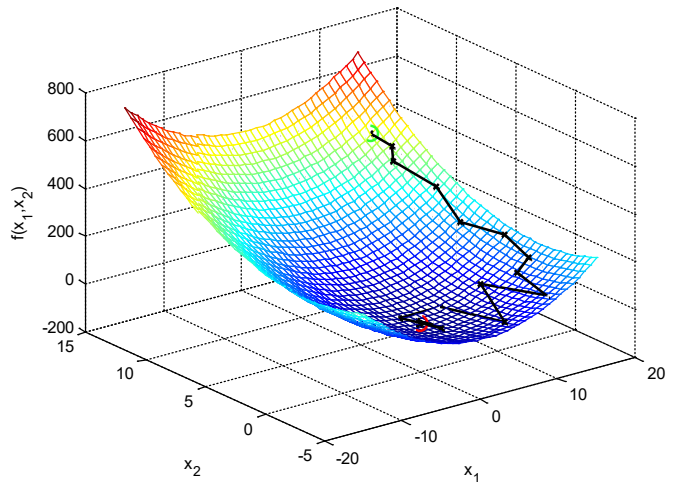
Schreiben Sie ein m-Skript, das, wie im Beispiel rechts gezeigt, den Funktionsverlauf von

$$f(x_1, x_2) = x_1^2 + 2 \cdot x_2^2 - x_1$$

(mit $x_i = [-12, 12]$)

in Abhängigkeit von x_1, x_2 darstellt und die Iterationsschritte einer Minimumsuche mit dem `fminsearch`-Befehl visualisiert.

Hinweis: `doc optimset` → `outputfcn`



Aufgabe 4.14: (ehemalige Klausuraufgabe)

Die Berechnung der Fourier-Reihe einer Funktion $y(t)$ beruht auf

$$C_N = \frac{1}{T} \int_{t_0}^{t_0+T} y(t) \cdot e^{-jn \frac{2\pi}{T} t} dt \quad (\text{komplexe Koeffizienten}).$$

Vervollständigen Sie (...) die M-Funktion `fourier` zur Berechnung des n -ten Fourier-Koeffizienten einer als Werte-Array `y` und Zeit-Array `t` über die Periode T gegebenen Funktion $y(t)$.

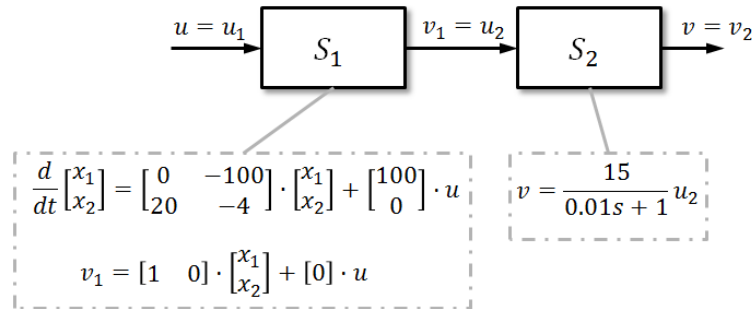
Beispielaufruf: (n = 1)

```
t=linspace(0,pi*2,200);
y = sin(t);
C1 = fourier(t,y,1)
```

```
function C = fourier(t,y,n)
    ...
```

Übung 5: Dynamische Systeme / Digitale Simulation / Simulink

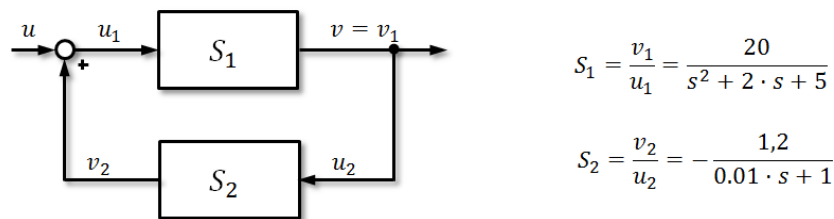
Aufgabe 5.1: Darstellung von LTI-Systemen



Gegeben sind zwei dynamische Systeme in den jeweils angegebenen Systemdarstellungen. Beide Systeme sind in Reihe geschaltet und es soll

- das Gesamtsystem als Übertragungsfunktion in Polynomdarstellung angegeben werden (was passiert, wenn Systeme in unterschiedlichen Darstellungen verknüpft werden?),
- die Stabilität des Gesamtsystems überprüft werden (die Polstellen weisen im stabilen Fall einen negativen Realteil auf) und
- das Gesamtsystem mit einer Abtastzeit von $T_s = 0.05s$ diskretisiert werden.

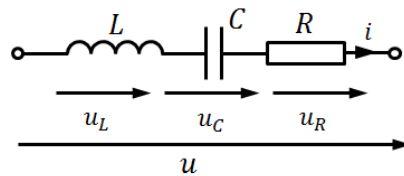
Aufgabe 5.2: Analyse von LTI-Systemen:



Gegeben ist die obige Kreisschaltung mit den Systemen S_1 und S_2 .

- Berechnen Sie das Gesamtsystem.
- Schreiben Sie ein Skript, welches für das Gesamtsystem die Nyquist-Ortskurve, den Pol-Nullstellenplan und das Bode-Diagramm in jeweils separate Fenster zeichnet und überprüfen sie die Stabilität des Systems.
- Zu dem System S_2 wird ein weiteres System $S_3 = -\frac{1}{1.2 \cdot (0.06 \cdot s + 1)}$ in Reihe geschaltet. Führen Sie die ersten zwei Punkte erneut durch und überprüfen Sie die Stabilität.
- Diskretisieren Sie das System mit $T_s = 0.1s$.
- Plotten Sie die Sprungantwort für das kontinuierliche und das diskrete System

Aufgabe 5.3: Lösen von ODEs:



$$\frac{di}{dt} = \frac{1}{L} \cdot (u - Ri - u_C)$$

$$\frac{du_C}{dt} = \frac{1}{C} \cdot i$$

Gegeben ist der oben dargestellte Reihenschwingkreis mit den zugehörigen Differenzialgleichungen. Es soll ein Skript geschrieben werden, das die Differenzialgleichungen mittels dem Befehl `ode23(...)` löst. Dabei stellt die Spannung u die Anregung des Systems dar und die Spannung u_C soll als Ausgang definiert werden. Die Anregung u ist eine Rechteckspannung mit der Amplitude $\hat{u} = 2V$ und einer Frequenz von $f = 10Hz$.

- Schreiben Sie ein Function-Handle, welcher die oberen beiden Differenzialgleichungen definiert (beachten Sie, dass die Solver mit variabler Schrittweite arbeiten und die Anregung interpoliert werden muss).
- Schreiben Sie ein Skript, welches die Anregung u erstellt (entweder manuell programmieren oder einen entsprechenden Befehl aus der Dokumentation suchen), die Funktion `ode23(...)` anwendet, und die Spannung u und u_C in einem Figure gegenüberstellt. Die Optionen für den Solver sollen alle auf Standard eingestellt werden. Die Anfangswerte werden zu Null angegeben und als Parameter der Differenzialgleichungen sollen folgenden Werte verwendet werden:

$$R = 100\Omega; \quad L = 10 H; \quad C = 500 \mu F$$

Aufgabe 5.4: (ehemalige Klausuraufgabe)

In der Komandozeile von Matlab soll der Aufruf

```
[z,p,k] = zpk_multi([1,2],[3,4],[4,6],[3,5],[0,6],[7,5]);
```

mit der folgenden Funktion `zpk_multi` vergleichbare Werte für z , p und k wie die M-Sequenz in Aufgabe 5.6 liefern. Dabei sollen die Befehle für Polynome und nicht die Befehle zur Erstellung von LTI-Objekten verwendet werden.

```
function [z, p, k] = zpk_multi(varargin)
    num = 1;
    den = 1;

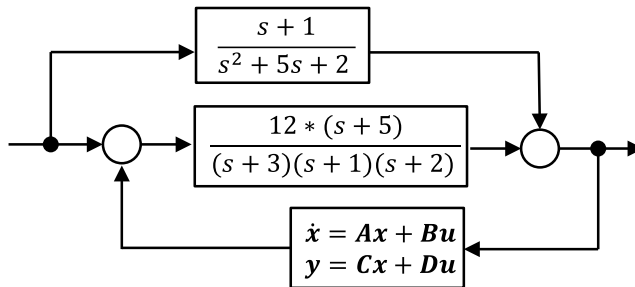
    for index = 1 : 2 : nargin
        ...
    end

    z = roots(num);
    p = roots(den);
    k = num(length(num)-length(z))/den(length(den)-length(p));
end
```

- Ergänzen Sie den Schleifenrumpf (...) so, dass die Funktion in der beschriebenen Weise funktioniert.
- Was unterscheidet die Ausgabe von der aus Aufgabe 4.5.

Aufgabe 5.5: (ehemalige Klausuraufgabe)

Gegeben ist folgendes dynamisches Gesamtsystem (Matrizen A, B, C und D gegeben):



- Erstellen Sie eine möglichst effiziente M-Sequenz zur Berechnung der Gesamtübertragungsfunktion und Berechnung der Nyquist-Ortskurve.
- Skizzieren Sie als Wirkungsplan das System, das durch das folgende LTI-Objekt `lti_summe` beschrieben wird:

```
lti1 = tf(1, [4, 2]);
lti2 = tf([1, 2], [2, 3]);
lti_summe = lti1 / (1 - lti1*lti2);
```

Aufgabe 5.6: (ehemalige Klausuraufgabe)

Beschreiben und begründen Sie die Funktion folgender M-Sequenz:

```
lti_tf      = tf([1, 2], [3, 4]) * tf([4, 6], [3, 5]) * tf([0, 6], [7, 5]);
lti_zpk     = zpk(lti_tf);
[z, p, k]   = zpkmdata(lti_zpk);
```

Hinweis: die Zahlwerte sind nur als Beispiel zu verstehen

Aufgabe 5.7: (ehemalige Klausuraufgabe)

Gegeben ist folgender Simulationsalgorithmus

$$\mathbf{x}_{k+1}^p = \mathbf{x}_k + T \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + T \left(\frac{5}{6} \mathbf{f}(\mathbf{x}_{k+1}^p, \mathbf{u}_{k+1}) + \frac{2}{6} \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) \right)$$

- Welchen fundamentalen Fehler enthält die Methode?
Führen Sie eine geeignete Korrektur durch.
- Wie würden Sie das Verfahren nach den Grundbegriffen der Simulationsverfahren klassifizieren?

Aufgabe 5.8: (ehemalige Klausuraufgabe)

Die Quadraturformel Simpson-Regel zur näherungsweisen Integration der Funktion $f(t)$ lautet

$$\int_a^b f(\tau) d\tau \approx (b-a) \cdot \left(\frac{1}{6} f(a) + \frac{4}{6} f\left(\frac{a+b}{2}\right) + \frac{1}{6} f(b) \right).$$

Formulieren Sie ein Verfahren mit dem „Expliziten Euler-Verfahren“ im ersten und zweiten (Prädiktor-Schritt) und der Simpson-Quadraturformel im letzten Schritt. Wie viele Stufen hat das Verfahren.

Aufgabe 5.9: (ähnlich ehemaliger Klausuraufgabe)

Ein gegebenes Runge-Kutta-Verfahren hat die Ordnung 5.

- Was bedeutet dies für die Abweichung des Simulationsergebnisses eines gegebenen dynamischen Systems von den idealen Werten nach der Simulationszeit τ .
- Hat das Verfahren einen größeren Stabilitätsbereich als Verfahren niedrigerer Ordnungen?

Aufgabe 5.10:

Erstellen Sie das Simulink-Modell eines mechanischen Masse-Feder-Dämpfer-System (s. Grafik rechts).

Eingangsgröße ist die Auslenkungskraft $F(t)$. Es soll die Auslenkung $y(t)$ simuliert werden.

DGL.:

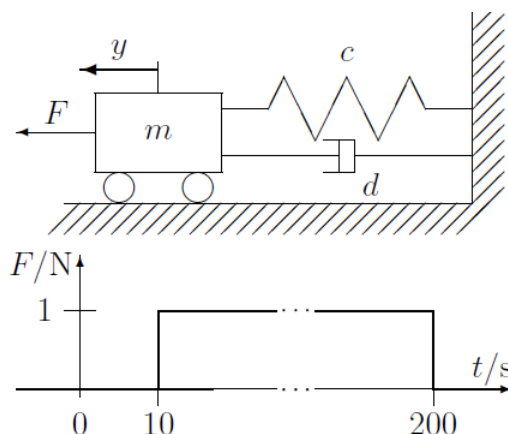
$$m\ddot{y} + d\dot{y} + cy = F \quad t > 0,$$

$$y(0) = y_0, \quad \dot{y}(0) = y_1$$

Parameter und Startwerte:

$$m = 10 \text{ kg}, \quad d = 1.0 \text{ Ns/m}, \\ c = 0.1 \text{ N/m}, \quad y_0 = 0 \text{ m}, \quad y_1 = 0 \text{ m/s}$$

Verwenden Sie für das Modell nur elementare Einzelblöcke (Integratoren, Gain, Addition, usw.).



Aufgabe 5.11: Subsysteme und Interaktion mit Matlab

Modifizieren Sie das Simulink-Modell aus Aufgabe 6.4 in folgender Weise:

- Mehrere Schwingungssysteme werden in separaten, parallelen Subsystemen mit unterschiedlichen Parametern simuliert, wobei die Ausgänge der Subsysteme in einem Bus zusammen geführt sind und das Ergebnis in einem Scope-Plot dargestellt wird.
- Die Parameter der Schwingungssysteme werden mittels eines M-Skripts initialisiert und die Simulationsergebnisse auf dem Matlab-Workspace abgelegt.

Aufgabe 5.12:

Erstellen Sie eine Simulink-Bibliothek mit dem Masse-Feder-Dämpfer-System aus Aufgabe 6.4 als Bibliothekselement:

- Statten Sie ihren Bibliotheksblock mit einer Maske mit Parametern, Beschreibungs- und Hilfetext und einem eigenen Layout aus.
- Nutzen Sie den Bibliotheksblock im Modell und erproben Sie die Bibliotheks-Mechanismen indem Sie den Link „disablen“, Änderungen im Modell vornehmen und den Link anschließend wiederherstellen.

Aufgabe 5.13:

Verwenden Sie das Modell des Masse-Feder-Dämpfer-System aus Aufgabe 6.4:

- Setzen Sie „Liniarization Points“ am Ein- und Ausgang des Modells. Starten Sie den „Control and Estimaton Tool Manager“ und stellen Sie die Eigenschaften des linearen Systems unterschiedlich dar: Bode-Diagramm, Nyquist-Ortskurve, usw.
- Diskretisieren Sie das Modell ($T = 2s$) und simulieren Sie erneut.

Aufgabe 5.14:

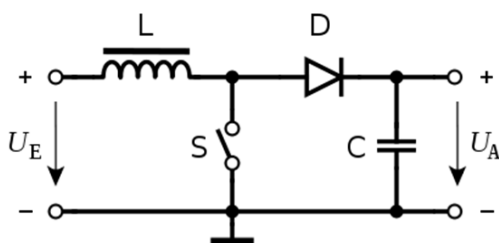
Schreiben Sie ein M-Skript, das das Simulationsmodell aus Aufgabe 6.4 so steuert, dass die Simulation mit sich ändernden Parametern (z. B. steigende Federsteifigkeit c) mehrfache automatisch gestartet wird und die Simulationsergebnisse anschließend in einer vergleichenden Grafik dargestellt werden.

Aufgabe 5.15: (ehemalige Klausuraufgabe)

Erklären Sie den Begriff des „Virtuellen Blocks“ in Simulink! Nennen Sie Beispiele! Sind alle Subsysteme „Virtuelle Blöcke“? Begründung!

Aufgabe 5.16:

Erstellen Sie in Simulink das Modell eines Hochsetzstellers (s. Grafik unten) mittels der Toolbox SimPowerSystems und Simulieren Sie das System.



- Eingangsspannung: $U_E = 200V$
- Induktivität: $L = 10mH$
- Ausgang (U_A) offen bzw. abgeschlossen mit einem Widerstand $R = 20\Omega$.
- Pulsweitenmodulation für den Schalter S: $f = 2 \text{ kHz}$, $d = 0..100 \%$

Aufgabe 5.17: Stabilität

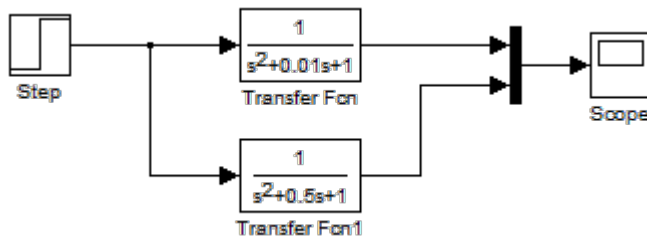
Wichtig: Bei allen folgenden Aufgabenteilen aus 6.1 geht es primär darum, die Ergebnisse zu vergleichen und plausibel zu erklären, nicht um die reine Ausführung der Simulation. Dieser Aufgabenpunkt ist im Folgenden nicht mehr explizit genannt.

a) Stabilitätsbereich I: Simulieren Sie folgendes Modell



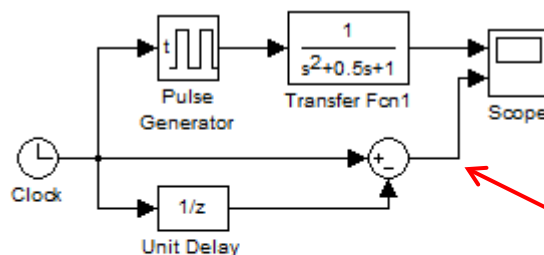
- mit dem expliziten Eulerverfahren und festen Schrittweiten 0.1 s; 1 s; 2 s; 3s und
- mit dem Runge-Kutta-4-Verfahren sowie dem Dormand-Prince-8-Verfahren und fester Schrittweite 3s.

b) Stabilitätsbereich II (schwingungsfähige Systeme): Simulieren Sie folgendes Modell



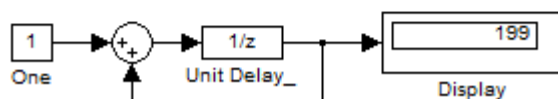
- mit dem expliziten Eulerverfahren und festen Schrittweiten (0.01 s; 0.1 s; 1 s) und
- mit dem Runge-Kutta-4-Verfahren und festen Schrittweiten (0.01 s; 0.1 s; 1 s).

c) Variable Schrittweite: Simulieren Sie mit folgendem Modell:



Simulationszeit: 200s
Pulsgenerator: $T_p = 200s$,
 $d = 50\%$

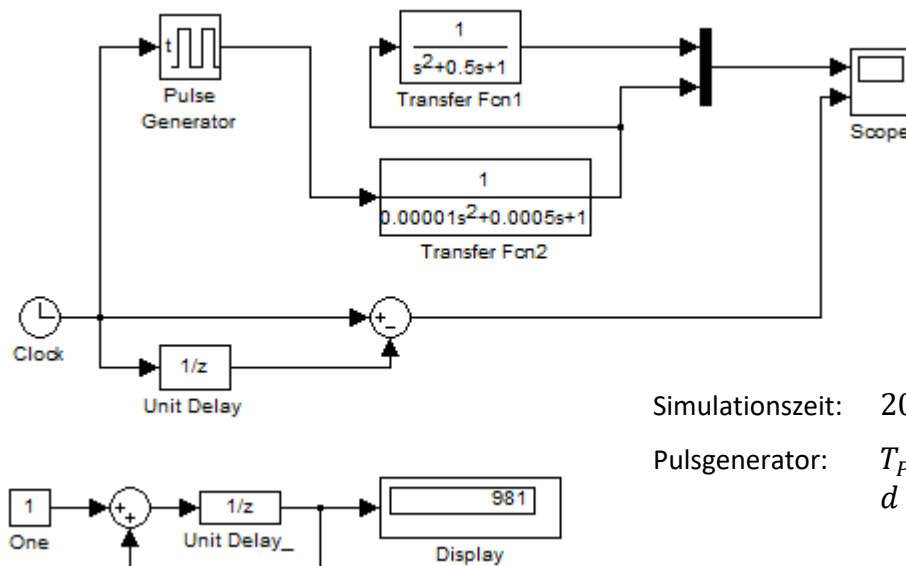
Ermittlung der Schrittweite



Ermittlung der Schrittzahl

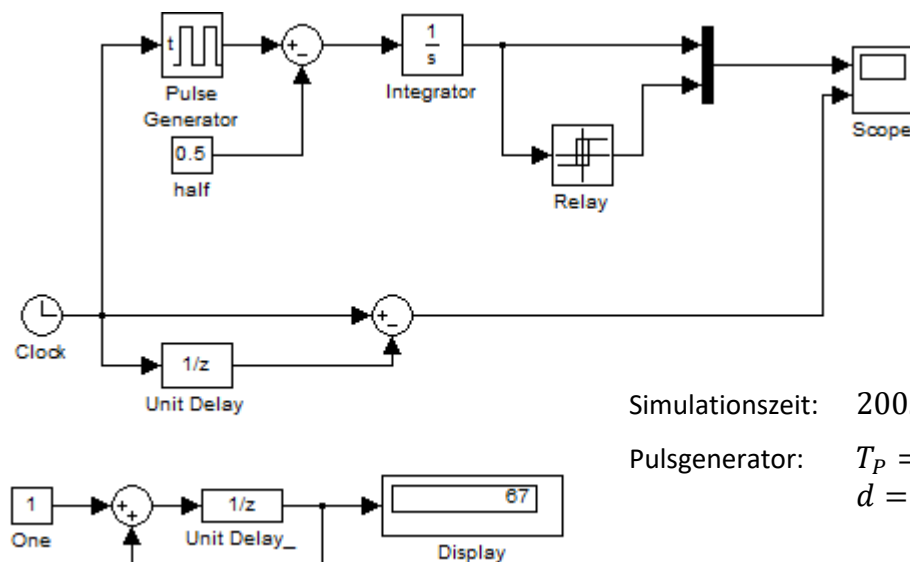
- Ermitteln Sie (experimentell und im Sekundenraster) die minimale mögliche feste Schrittweite beim Runge-Kutta-4-Verfahren für eine stabile Simulation.
- Simulieren Sie mit variabler Schrittweite und Dormand-Prince (ode45) und vergleichen Sie die Ergebnisse.
- Variieren Sie die Einstellungen für die Simulation mit variabler Schrittweite.

d) Steife Systeme: Simulieren Sie folgendes Modell:



- Ermitteln Sie (experimentell und im Sekundenraster) die minimale mögliche feste Schrittweite beim Runge-Kutta-4-Verfahren für eine stabile Simulation.
- Simulieren Sie mit variabler Schrittweite und dem Dormand-Prince Verf. (ode45).
- Simulieren Sie mit variabler Schrittweite und einem Verfahren für steife Systeme.

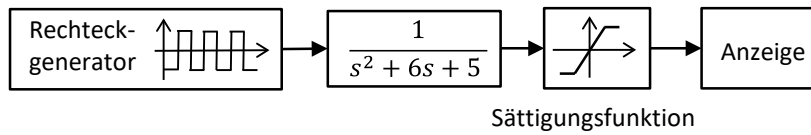
e) Zero-Crossing-Detection: Simulieren Sie folgendes Modell:



- Simulieren Sie mit variabler Schrittweite und dem Dormand-Prince Verf. (ode45).
- Deaktivieren Sie die Zero-Crossing-Detection und simulieren Sie erneut.

Aufgabe 5.18: (ehemalige Klausuraufgabe)

Gegeben ist folgendes dynamisches System:



- Das System soll mit dem „Expliziten Euler-Verfahren“ simuliert werden. Welche Abtastzeit wählen Sie? Begründung!
- Würden Sie ggf. ein Verfahren höher Ordnung wählen? Begründen Sie Ihre Antwort!
- Würden Sie ggf. ein Mehrschrittverfahren wählen? Begründen Sie Ihre Antwort!
- Die Übertragungsfunktion lautet jetzt

$$G(s) = \frac{1}{s^2 + 0.001s + 1} \quad .$$

Was ist bzgl. des Simulationsalgorithmus zu beachten.